GNU – GESTION DES NOTES UNIVERSITAIRES

Guide de recette

API, Postman et parcours des trois acteurs

Guide pratique pour préparer, exécuter et consigner les tests locaux

Version du guide :	1.0 – 15 octobre 2026
API locale :	`http://127.0.0.1:8000/api/v1`
Interface locale :	`http://127.0.0.1:8780`
Base de test :	PostgreSQL local, base `gnu_notes`

Ce guide décrit les routes présentes dans le dépôt au jour de rédaction.

Ne pas exécuter de scénario d'écriture ou de publication sur une base réelle.

Table des matières

1 Comment utiliser ce guide 1

2 Comprendre GNU avant de commencer 1

3 Préparer la base et les comptes 2

4 Démarrer l'API et l'interface 4

5 Configurer Postman 5

6 Comprendre les réponses HTTP 6

7 Parcours 1 – Agent 6

8 Parcours 2 – Enseignant 9

9 Parcours 3 – Étudiant 10

10 Tester l'application dans le navigateur 11

11 Tests d'autorisation et de cohérence 12

12 Catalogue synthétique des routes 12

13 Règles métier à vérifier pendant la recette 14

14 Plan de recette et critères d'acceptation 14

15 Fiche d'exécution à dupliquer 17

16 Dépannage rapide 18

17 Construire la documentation utilisateur d'environ 50 pages 18

18 Sources techniques vérifiées 18

1 Comment utiliser ce guide

Ce document accompagne une recette fonctionnelle de GNU. Il explique comment démarrer l'environnement local, préparer Postman, s'authentifier avec chaque rôle, tester les parcours Agent, Enseignant et Étudiant, puis consigner les résultats. Il ne prétend pas que les scénarios ont déjà été exécutés avec succès.

Pour chaque essai, utiliser des comptes et des données de test identifiables. Noter la date, la version du dépôt, les préconditions, l'action envoyée, la réponse reçue et une référence de preuve. En cas d'échec, garder le code HTTP et le message d'erreur, mais supprimer les mots de passe, jetons et données personnelles des captures.

> **À retenir.** Les commandes de ce guide visent la base locale sur le port 5435. Le seeder réinitialise les mots de passe des trois comptes de démonstration à chaque exécution. Ne pas l'exécuter contre une base de production.

2 Comprendre GNU avant de commencer

GNU est composé de trois éléments qui doivent fonctionner en même temps :

1. **PostgreSQL** garde les données : utilisateurs, départements, classes, UE, inscriptions, notes et bulletins.
2. **L'API Laravel** reçoit les demandes et applique les règles d'accès et de calcul. Elle tourne ici à `http://127.0.0.1:8000`.
3. **L'application Flutter Web** est l'interface que l'utilisateur voit dans le navigateur. Elle tourne ici à `http://127.0.0.1:8780`.

Postman est un outil séparé qui envoie à l'API les mêmes demandes que l'interface, mais sans écran GNU. Il permet de vérifier le résultat brut, le code HTTP et le corps JSON. Postman ne remplace pas les tests visuels : il faut tester les deux.

Dans une URL Postman, la méthode indique l'action : `GET` lit, `POST` crée/transmet, `PUT` enregistre/remplace, `PATCH` modifie un statut. Un Bearer token est une clé temporaire obtenue après connexion. Une réponse JSON ressemble à un objet avec des champs, par exemple `data`, `id` et `message`.

2.1 Ce qu'il faut avoir installé

Les commandes de ce guide visent Linux/Ubuntu et le dépôt local. Ouvrir l'application **Terminal** depuis le menu du système. Dans le terminal, coller une commande puis appuyer sur Entrée. Lorsque `sudo` le demande, saisir le mot de passe de session Linux : les caractères ne s'affichent pas pendant la saisie, c'est normal.

GNU nécessite PHP avec l'extension PostgreSQL, Composer, PostgreSQL, Flutter/Dart et un navigateur. Dans le terminal, vérifier :

```
php -v
composer --version
php -m
psql --version
flutter --version
```

Si une commande dit `command not found`, l'outil correspondant n'est pas installé ou n'est pas dans le `PATH`. Ne pas continuer au hasard : noter la commande manquante et installer le prérequis de développement sur cette machine. Dans `php -m`, vérifier la présence de `pdo_pgsql`.

2.2 Une nouvelle installation est différente d'un redémarrage

Pour redémarrer le GNU déjà configuré sur cet ordinateur, suivre les sections « Vérifier la base » puis « Démarrer ». Le dépôt contient les migrations Laravel récentes, mais **ne contient pas le fichier SQL complet qui crée initialement toutes les tables du schéma GNU**. Il contient des scripts d'import partiels, qui ne remplacent pas ce schéma. Une base vide ne peut donc pas être construite avec la seule commande `php artisan migrate`.

Pour installer sur un ordinateur neuf, il faut aussi le DDL de base ou une sauvegarde complète de la base provenant de l'administrateur. Sans cette source, arrêter après l'installation des logiciels : ne pas créer une fausse base vide, ne pas exécuter les imports partiels comme s'ils créaient le modèle entier. Ce guide utilise la base locale `gnu_notes` déjà préparée et ses migrations récentes.

2.3 Protection des données pendant les essais

Les essais de création, de saisie de note, de décision et de publication écrivent dans la base. Si elle contient les vrais étudiants, ne pas réaliser ces scénarios dedans. Créer une copie locale de recette ou demander une base dédiée. Un bulletin publié est une version historique ; une note retirée est conservée en historique ; une décision d'inscription change l'état du dossier.

Pour copier la base locale sur le même serveur PostgreSQL, si le rôle PostgreSQL permet le dump et si le nom `gnu_notes_recette` n'existe pas déjà :

```
sudo -u postgres pg_dump -p 5435 -d gnu_notes -Fc -f /tmp/gnu_notes_recette.dump
sudo -u postgres createdb -p 5435 gnu_notes_recette
sudo -u postgres pg_restore -p 5435 -d gnu_notes_recette --no-owner /tmp/gnu_notes_recette.dump
```

Dans `backend/.env`, remplacer uniquement la valeur de `DB_DATABASE` par le nom de la copie. Dans le dossier `backend`, lancer :

```
php artisan config:clear
```

Si la copie échoue ou si la base cible existe déjà, ne pas supprimer de base pour contourner l'erreur : demander de l'aide avant toute commande de suppression. Restaurer une sauvegarde peut copier des données personnelles ; la copie reste locale et confidentielle.

Si tu utilises cette copie, remplacer aussi `-d gnu_notes` par `-d gnu_notes_recette` dans toutes les commandes `psql` ci-dessous afin de ne pas interroger/modifier la base source.

2.4 État fonctionnel décrit

L'application dispose de routes d'authentification et de portails par rôle. L'Agent peut travailler sur son périmètre départemental : classes, catalogue UE/EC, comptes visibles, affectations, inscriptions, requêtes, rattrapages, aperçus de bulletins, publications autorisées et audit. L'Enseignant dispose du plan d'évaluation, du carnet, de la saisie/import/transmission de notes, des bulletins provisoires et du traitement des requêtes affectées. L'Étudiant dispose des inscriptions pédagogiques, résultats provisoires et publiés, requêtes, justificatifs, rattrapages, notifications et profil.

La démonstration par seeder ne crée pas l'ensemble des données pédagogiques. Le professeur n'a pas d'affectation par défaut ; l'Agent n'est pas rattaché à un département ; le compte étudiant peut ne pas disposer d'une inscription de classe. Il faut vérifier et préparer ces données avant d'essayer ces scénarios.

3 Préparer la base et les comptes

3.1 Vérifier la connexion et les migrations

Dans un terminal :

```
sudo service postgresql start
pg_isready -h 127.0.0.1 -p 5435
cd "/home/agent-47/Téléchargements/GNU---Gestion-des-notes-Universitaires/backend"
php artisan migrate:status
```

Vérifier que PostgreSQL accepte les connexions et que les migrations attendues apparaissent `Ran`, dont `2026_10_15_000001`, `000002` et `000003`. Si une migration est encore `Pending`, ne pas lancer de tests qui modifient des données avant de comprendre pourquoi.

Dans `backend/.env`, vérifier les paramètres locaux (ne pas les publier) : `APP_ENV=local`, `DB_HOST=127.0.0.1`, `DB_PORT=5435`, `DB_DATABASE=gnu_notes`. Le compte PostgreSQL doit être celui utilisé localement par l'API.

3.2 Vérifier le compte Agent

La commande suivante affiche uniquement l'identifiant technique, le login, le rôle et l'état du compte :

```
sudo -u postgres psql -X -p 5435 -d gnu_notes -c \
"SELECT id, login, role, actif FROM gnu.utilisateur WHERE login = 'ADM-8842-CELLULE';"
```

Une ligne `AGENT` active confirme l'existence du compte. Le mot de passe est illisible dans la base ; il faut retrouver la valeur locale de `GNU_DEMO_PASSWORD`. Le code du dépôt ne fournit pas de création Agent en libre-service. La commande Artisan `app:create-agent` est un squelette non implémenté.

3.3 Créer ou réinitialiser les profils de démonstration

Dans le fichier local `backend/.env`, configurer un mot de passe d'essai d'au moins 12 caractères :

```
GNU_ALLOW_DEMO_SEED=true
GNU_DEMO_PASSWORD=Choisir-Un-Mot-De-Passe-Local-Long
```

Puis exécuter :

```
php artisan config:clear
php artisan db:seed --class=DemoGnuAccountsSeeder --no-interaction
```

Identifiants créés ou réactivés :

Rôle	Login de démonstration
Étudiant	`21T2355`
Enseignant	`ENS-4182-INFO`
Agent	`ADM-8842-CELLULE`

Les trois partagent le mot de passe local configuré. Chaque exécution du seeder réinitialise les trois mots de passe. Le seeder n'ajoute ni classe, ni inscription, ni affectation, ni rattachement départemental de l'Agent.

3.4 Attribuer le département Informatique à l'Agent

L'API Agent filtre les lectures et écritures par les lignes de `gnu.agent_departement`. D'abord relever les identifiants exacts :

```
sudo -u postgres psql -X -p 5435 -d gnu_notes -c \
"SELECT id, nom_depart FROM gnu.departement ORDER BY nom_depart;"
sudo -u postgres psql -X -p 5435 -d gnu_notes -c \
"SELECT id, login, role FROM gnu.utilisateur WHERE login = 'ADM-8842-CELLULE';"
```

Après vérification visuelle des deux lignes, remplacer `ID_AGENT` et `ID_DEPARTEMENT` par les nombres retournés :

```
sudo -u postgres psql -X -v ON_ERROR_STOP=1 -p 5435 -d gnu_notes -c \
"INSERT INTO gnu.agent_departement (agent_id, departement_id) \
VALUES (ID_AGENT, ID_DEPARTEMENT) ON CONFLICT DO NOTHING;"
```

Contrôler le rattachement :

```
sudo -u postgres psql -X -p 5435 -d gnu_notes -c \
"SELECT u.login, d.nom_depart FROM gnu.agent_departement ad \
JOIN gnu.utilisateur u ON u.id = ad.agent_id \
JOIN gnu.departement d ON d.id = ad.departement_id \
WHERE u.login = 'ADM-8842-CELLULE';"
```

Le tableau de bord doit alors compter les données de ce périmètre. Sans ligne de rattachement, il peut afficher des métriques vides ; ce n'est pas une panne d'authentification.

> **À retenir.** Les nombres d'exemple dans les corps JSON (par exemple `niveau_id=12`, `classe_id=42` ou `enseignant_id=3`) ne sont pas garantis dans ta base. Remplacer chaque nombre par l'ID exact obtenu depuis une route `GET`. Ne jamais deviner un ID.

3.5 Vérifier que les comptes ont des données pour les scénarios

Un compte peut se connecter sans avoir une classe, une inscription ou un cours à tester. Les commandes ci-dessous sont des lectures seulement et n'affichent pas de mot de passe :

```
sudo -u postgres psql -X -p 5435 -d gnu_notes -c "SELECT u.login, e.matricule, c.code_classe, ic.
    statut_inscription, ic.id AS inscription_classe_id FROM gnu.utilisateur u JOIN gnu.etudiant e ON e.
    utilisateur_id = u.id LEFT JOIN gnu.inscription_classe ic ON ic.etudiant_id = e.id LEFT JOIN gnu.classe c
    ON c.id = ic.classe_id WHERE u.login = '21T2355';"
```

```
sudo -u postgres psql -X -p 5435 -d gnu_notes -c "SELECT u.login, e.eid, c.code_classe, m.code_m, a.id AS
     affectation_id, a.actif FROM gnu.utilisateur u JOIN gnu.enseignant e ON e.utilisateur_id = u.id LEFT JOIN
     gnu.affectation a ON a.enseignant_id = e.id LEFT JOIN gnu.classe c ON c.id = a.classe_id LEFT JOIN gnu.
     matiere m ON m.id = a.matiere_id WHERE u.login = 'ENS-4182-INFO';"
sudo -u postgres psql -X -p 5435 -d gnu_notes -c "SELECT u.login, d.nom_depart FROM gnu.utilisateur u LEFT JOIN
      gnu.agent_departement ad ON ad.agent_id = u.id LEFT JOIN gnu.departement d ON d.id = ad.departement_id
     WHERE u.login = 'ADM-8842-CELLULE';"
```

Si l'étudiant n'a pas de classe, l'enseignant aucune affectation, ou l'Agent aucun département, leurs écrans de rôle peuvent s'ouvrir mais les scénarios correspondants seront vides. Le seeder ne crée pas ces relations. Il faut préparer un jeu d'essai cohérent dans une base de recette par l'interface Agent ou avec l'administrateur de la base avant les parcours complets ; ne pas inventer manuellement des inscriptions ou des notes dans la base réelle.

4 Démarrer l'API et l'interface

4.1 Préparer le backend la première fois seulement

Dans un terminal, entrer dans le dossier du backend :

```
cd "/home/agent-47/Téléchargements/GNU---Gestion-des-notes-Universitaires/backend"
composer install
```

Composer installe les bibliothèques PHP du projet. Cela peut nécessiter Internet et prendre quelques minutes. Si `vendor/` est déjà présent et la commande se termine sans erreur, cette étape n'est pas à répéter à chaque démarrage.

Si le fichier `.env` n'existe pas, le créer une seule fois à partir de l'exemple :

```
cp .env.example .env
php artisan key:generate
```

Ouvrir ensuite `backend/.env` dans un éditeur de texte et vérifier les valeurs locales :

```
APP_ENV=local
APP_DEBUG=false
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5435
DB_DATABASE=gnu_notes
DB_USERNAME=gnu_app
DB_PASSWORD=mot_de_passe_postgres_local
CORS_ALLOWED_ORIGINS=http://127.0.0.1:8780,http://localhost:8780
```

Remplacer la valeur d'exemple de `DB_PASSWORD` par celle de la configuration locale. Ne pas écraser un fichier `.env` déjà fonctionnel avec le modèle et ne jamais partager son contenu : il contient des secrets.

Terminal 1 – API Laravel :

```
cd "/home/agent-47/Téléchargements/GNU---Gestion-des-notes-Universitaires/backend"
php artisan serve --host=127.0.0.1 --port=8000
```

Terminal 2 – Flutter Web :

```
cd "/home/agent-47/Téléchargements/GNU---Gestion-des-notes-Universitaires/frontend"
flutter pub get
flutter run -d web-server --web-hostname 127.0.0.1 --web-port 8780 \
  --dart-define=GNU_API_BASE_URL=http://127.0.0.1:8000/api/v1
```

Ouvrir `http://127.0.0.1:8780`. Le portail demande de choisir l'un des trois espaces puis les identifiants. Postman appelle directement l'API à `http://127.0.0.1:8000/api/v1` ; l'interface et Postman sont deux clients du même backend.

4.2 Arrêter les services

Quand les essais sont terminés, revenir à chaque terminal serveur et appuyer sur **Ctrl+C**. Cela arrête le serveur Flutter ou Laravel en cours. Ne pas arrêter PostgreSQL s'il est utilisé par d'autres applications.

5 Configurer Postman

5.1 Créer l'environnement

Dans Postman, créer un environnement nommé **GNU Local** et y ajouter les variables suivantes. Marquer le mot de passe et le jeton comme sensibles/privés si la version de Postman le permet.

Variable	Valeur initiale	Type conseillé
`baseUrl`	`http://127.0.0.1:8000/api/v1`	default
`login`	login du rôle choisi	local
`password`	mot de passe local du seeder	secret/local
`token`	vide	secret/local
`agentToken`, `teacherToken`, `studentToken`	vides	secret/local
`classId`, `levelId`, `yearId`	vides au départ	local
`assignmentId`, `evaluationId`, `noteId`	vides au départ	local
`unitId`, `ueCode`, `requestId`	vides au départ	local

Ne jamais exporter une collection avec un jeton ou mot de passe actif. Ne pas mettre les valeurs réelles dans l'onglet *Initial value* synchronisé ; utiliser l'onglet local/current value ou un vault prévu à cet effet.

5.2 Créer une collection et les en-têtes

Créer une collection **GNU API - recette locale**. Pour les dossiers nécessitant connexion, configurer **Authorization > Bearer Token > Token** à `{{token}}`. Pour les corps JSON, ajouter :

```
Accept: application/json
Content-Type: application/json
```

Pour les téléchargements de fichiers, Postman doit générer lui-même le `Content-Type` multipart ; ne pas le fixer manuellement.

5.3 Faire sa première demande dans Postman

1. Ouvrir Postman. Dans le sélecteur d'environnement près du bouton d'envoi, choisir **GNU Local**. 2. Choisir **New > HTTP Request** (ou créer une demande depuis la collection). Nommer la demande **Connexion Agent** et la sauvegarder dans **GNU API - recette locale > Authentification**. 3. Dans la liste de méthode, choisir **POST**. Dans le champ d'adresse, saisir `{{baseUrl}}/auth/login`. 4. Ouvrir l'onglet **Headers** ; ajouter `Accept: application/json`. Ouvrir **Body > raw**, choisir **JSON**, puis coller l'objet de connexion ci-dessous. 5. Cliquer **Send**. Sous la demande, regarder le code HTTP, le temps de réponse et l'onglet **Body**. Si le statut est 200, vérifier `utilisateur.role`. 6. Si le statut est 401, vérifier le login, le mot de passe local, le rôle choisi et l'existence/activation du compte. Si c'est une erreur réseau, vérifier d'abord que le terminal API est toujours ouvert.

Les autres demandes se créent de la même manière : choisir la méthode, saisir l'URL, sélectionner le type d'autorisation et le corps demandé par le scénario, puis cliquer **Send**. Pour lire une route, utiliser **GET** ; pour un corps JSON, choisir **Body > raw > JSON** ; pour joindre un fichier, choisir **Body > form-data** et transformer la valeur de `fichier` en type **File**.

> **À retenir.** Le dépôt contient un ancien fichier `backend/GNU_Postman_Collection.json`. Il inclut la route `/api/v1/comptes`, qui n'existe pas dans l'API actuelle. Pour suivre ce guide, créer une nouvelle collection ou vérifier chaque adresse avec le catalogue actuel avant d'envoyer.

5.4 Requête de connexion et stockage du jeton

Créer `POST {{baseUrl}}/auth/login`, corps **raw / JSON** :

```
{
  "login": "{{login}}",
  "password": "{{password}}",
  "device_name": "postman-recette-local"
}
```

Dans l'onglet **Scripts > Post-response**, ajouter :

```
pm.test("Connexion reussie", function () {
  pm.response.to.have.status(200);
});
const body = pm.response.json();
pm.environment.set("token", body.token);
pm.environment.set("roleConnecte", body.utilisateur.role);
if (body.utilisateur.role === "AGENT") pm.environment.set("agentToken", body.token);
if (body.utilisateur.role === "ENSEIGNANT") pm.environment.set("teacherToken", body.token);
if (body.utilisateur.role === "ETUDIANT") pm.environment.set("studentToken", body.token);
```

Exécuter cette demande une fois par acteur ; chaque connexion remplit le jeton correspondant et remplace la variable active `token`. Les routes protégées utiliseront alors `Authorization: Bearer <token>`.

> **À retenir.** Un jeton est un secret d'accès à durée limitée. L'API retourne un jeton Bearer et une date d'expiration. Ne pas envoyer de jeton par message et ne pas l'inclure dans les captures de Postman.

5.5 Contrôles de base à automatiser

Dans les demandes de lecture habituelles, vérifier le code attendu et le type JSON :

```
pm.test("Code HTTP attendu", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 201, 204]);
});
if (pm.response.code !== 204) {
  pm.test("Reponse JSON", function () {
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
  });
}
```

Ne pas imposer le même code à toutes les routes : une création réussie retourne souvent `201`, une suppression de session `204`, et un rejet de validation `422`.

6 Comprendre les réponses HTTP

Code	Interprétation à vérifier
200	Lecture ou mutation réussie avec contenu.
201	Ressource créée (classe, affectation, session, etc.).
204	Action réussie sans corps, par exemple déconnexion.
401	Pas de jeton valide, identifiants erronés ou session expirée.
403	Jeton valide mais rôle/habilitation insuffisant, notamment publication sans habilitation.
404	Identifiant inexistant ou hors du périmètre permis.
409	Conflit d'état/version : transition interdite, note modifiée, inscription déjà traitée, publication existante.
422	Corps/champ invalide, note hors intervalle ou relation incompatible.
429	Limitation de fréquence atteinte (connexion ou API).
503	Fonction temporairement verrouillée par configuration, notamment publication officielle.

Les contrôles d'erreur font partie de la recette. Ne pas considérer une requête HTTP qui retourne 4xx comme un défaut automatiquement : comparer le statut, le message et l'effet en base au cas testé.

7 Parcours 1 – Agent

7.1 Authentification, profil et tableau de bord

1. Choisir Agent, exécuter `POST /auth/login` avec `ADM-8842-CELLULE`.
2. Vérifier `200`, `token_type=Bearer` et `utilisateur.role=AGENT`.
3. Exécuter `GET /auth/me` avec Bearer token ; vérifier l'identité du compte.

4. Exécuter `GET /agent/dashboard` ; relever les compteurs, dont `departements`, `classes` et `peut_publier`.
5. Exécuter `GET /agent/classes`, `GET /agent/niveaux` et `GET /agent/catalogue` ; confirmer le périmètre Informatique.

Demande	Attendu
`GET /agent/dashboard`	`200` ; les compteurs correspondent au département rattaché.
`GET /agent/classes`	`200` ; classe, niveau, filière, année et version disponibles.
`GET /agent/niveaux`	`200` ; niveaux des filières du périmètre uniquement.
`GET /agent/catalogue`	`200` ; clés `ues` et `elements_constitutifs`.
`GET /agent/comptes`	`200` ; utilisateurs liés aux inscriptions/affectations du département.
`GET /agent/affectations`	`200` ; enseignant, EC, UE et classe liés.
`GET /agent/inscriptions`	`200` ; inscriptions du département, éventuellement filtrées par `?statut=SOUMISE`.
`GET /agent/requetes`	`200` ; requêtes soumises/en traitement du département.
`GET /agent/rattrapages/sessions`	`200` ; sessions du département.
`GET /agent/audit?limit=100`	`200` ; maximum de 250 entrées dans le périmètre.

Pour une requête non authentifiée, refaire temporairement `GET /agent/dashboard` sans Bearer : l'API doit répondre `401`. Refaire une route Agent avec le jeton Étudiant : l'API doit refuser l'accès (rôle insuffisant).

7.2 Créer une classe de test

Trouver le `niveau_id` avec la lecture suivante, puis relever l'année via les référentiels authentifiés :

```
GET {{baseUrl}}/agent/niveaux
GET {{baseUrl}}/referentiels/annees-academiques
```

Créer une classe dédiée :

```
{
  "code_classe": "TEST-INF-L3-2026",
  "niveau_id": 12,
  "annee_id": 4,
  "version_programme": 1
}
```

Remplacer les identifiants par les valeurs retournées dans ta base. Attendu : `201`, réponse avec `id` et message. Conserver cet ID dans `classId` ; relire la liste des classes. Si la classe existe déjà, choisir un code inédit plutôt que de renvoyer la création.

7.3 Créer une UE et ses EC de test

Créer une UE en brouillon par `POST /agent/ues` :

```
{
  "code_ue": "TEST-UE-INF",
  "intitule_ue": "UE de recette API",
  "niveau_id": 12,
  "version_programme": 1,
  "numero_semestre": 1,
  "categorie_ue": "FONDAMENTALE",
  "credit_ue": 4
}
```

Attendu : `201`, UE `BROUILLON`. Ajouter de un à deux éléments constitutifs avec `POST /agent/ues/{ue}/elements` :

```
{
  "code_m": "TEST-EC-INF",
  "intitule_matiere": "Élément de recette",
  "rang_ec": 1,
  "credit_matiere": 4
}
```

Pour activer l'UE, `PATCH /agent/ues/{ue}/statut` avec `{"statut_catalogue":"ACTIF"}`. Elle doit comporter un ou deux EC. Après le test, archiver avec `ARCHIVE` si aucune autre donnée ne l'utilise. Les codes et IDs doivent être conservés dans les variables Postman.

7.4 Affecter un enseignant

Relever `enseignant_id` et `matiere_id` dans les réponses de comptes/catalogue. Le compte enseignant doit être actif, l'EC dans le programme actif du niveau et version choisis. Envoyer `POST /agent/affectations` :

```
{
  "enseignant_id": 3,
  "matiere_id": 18,
  "classe_id": 42
}
```

Attendu : `201`. En cas de classe/EC hors périmètre : refus. En cas d'enseignant inactif ou de matière incompatible : `422`. Vérifier ensuite `GET /agent/affectations`, puis se connecter avec le rôle Enseignant et lire `GET /enseignant/affectations`.

7.5 Examiner inscription et transmettre une requête

Les opérations suivantes nécessitent des objets existants à l'état attendu :

— **Inscription** : lire `GET /agent/inscriptions?statut=SOUMISE`, puis envoyer :

```
PATCH {{baseUrl}}/agent/inscriptions/{{enrollmentId}}
{"decision":"VALIDER", "motif":"Dossier de test complet et vérifié"}
```

Réponse attendue : `200` ; l'état devient `VALIDEE`.

— **Refus** : même route avec `decision=REFUSER` et un motif d'au moins huit caractères. L'état devient `REFUSEE`. N'utiliser qu'une inscription de test, car seule une inscription `SOUMISE` peut être examinée.

— **Requête** : relever l'enseignant affecté et `requete_id` dans `GET /agent/requetes`. Le routage se fait par :

```
PATCH {{baseUrl}}/agent/requetes/{{requestId}}/affectation
{"enseignant_id":3, "motif":"Transmission au professeur affecté pour examen"}
```

Seul l'enseignant actif affecté à cette classe et cet EC est accepté. **Limite constatée dans le code :** la route de dépôt Étudiant enregistre actuellement `matiere_id=null`, alors que le routage Agent exige un EC précis. Le routage de ces requêtes est donc bloqué de bout en bout. Consigner ce scénario comme **BLOQUÉ / anomalie fonctionnelle connue**.

7.6 Sessions de rattrapage

Créer avec `POST /agent/rattrapages/sessions` :

```
{
  "classe_id": 42,
  "libelle": "Rattrapage de recette",
  "date_session": "2026-12-15"
}
```

La date doit être aujourd'hui ou ultérieure. Attendu : `201`, statut initial `PREVUE`. Faire évoluer avec `PATCH /agent/rattrapages/sessions/{id}` : `PREVUE -> OUVERTE -> CLOTUREE`. Tester aussi l'annulation d'une session prévue. Une transition non permise doit retourner `409`.

7.7 Aperçu MGP et publication

1. Depuis `GET /agent/classes`, utiliser l'ID de la classe de test.
2. Lire cet aperçu (périodes : `S1`, `S2`, `ANNUELLE`) :

   ```
   GET {{baseUrl}}/agent/classes/{{classId}}/apercu-bulletins?periode=S1
   ```

3. Vérifier `pret`, chaque étudiant, le calcul MGP, les notes et les valeurs `EL` en cas de note manquante.

4. Vérifier `peut_publier`. S'il vaut `false`, ne pas tenter de publication : l'habilitation doit être donnée séparément par l'administration.

La route de publication exige l'habilitation distincte, le paramètre `GNU_ALLOW_OFFICIAL_PUBLICATION=true` et des résultats calculables :

```
POST {{baseUrl}}/agent/classes/{{classId}}/publications
{"periode":"S1"}
```

Une réussite crée une publication et fige un bulletin. Une deuxième publication de la même période renvoie `409` à moins que `confirmer_nouvelle_version=true`. Ne tester cette route que sur des données explicitement dédiées à la recette. Pour les tests ordinaires, s'arrêter à l'aperçu provisoire.

8 Parcours 2 – Enseignant

8.1 Affectations et plan d'évaluation

Avec le jeton Enseignant, appeler `GET /enseignant/affectations`. Sélectionner l'`affectation_id` obtenue. Le compte de démonstration n'ayant pas d'affectation par défaut, effectuer d'abord le test Agent précédent.

Lire le plan, puis enregistrer une version :

```
GET {{baseUrl}}/enseignant/affectations/{{assignmentId}}/plan-evaluation
PUT {{baseUrl}}/enseignant/affectations/{{assignmentId}}/plan-evaluation
```

```
{
  "motif": "Plan de recette validé pour la classe de test",
  "evaluations": [
    {"type_eval":"CC", "date_evaluation":"2026-11-02", "poids":30},
    {"type_eval":"SN", "date_evaluation":"2026-12-03", "poids":70}
  ]
}
```

Le total des poids doit être exactement 100. Pour un test avec TP, utiliser par exemple CC 20, TP 30, SN 50. Une somme invalide doit être rejetée. La création retourne `201` et un nouveau plan versionné.

8.2 Carnet, statut d'évaluation et saisie des notes

Appeler `GET /enseignant/affectations/{id}/carnet` ; relever les IDs d'évaluation et les matricules des inscriptions validées. Avant de saisir des notes, passer l'évaluation à `REALISEE` :

```
PATCH {{baseUrl}}/enseignant/affectations/{{assignmentId}}/evaluations/{{evaluationId}}
{
  "statut_evaluation": "REALISEE",
  "motif": "Épreuve terminée pour le test local"
}
```

Enregistrer les notes avec `PUT /enseignant/affectations/{id}/evaluations/{id}/notes` :

```
{
  "motif": "Saisie initiale des notes de recette",
  "notes": [
    {"matricule":"21T2355", "situation":"NUMERIQUE", "valeur_note":14.5},
    {"matricule":"ETU-TEST-02", "situation":"ABSENTE", "valeur_note":null},
    {"matricule":"ETU-TEST-03", "situation":"MANQUANTE", "valeur_note":null}
  ]
}
```

Notes numériques : 0 à 20. Une absence ou valeur manquante ne reçoit pas de note numérique. Le champ `version_note` est optionnel à la première création et requis pour les modifications avec contrôle de version ; recharger le carnet pour récupérer la version courante. Le matricule doit être inscrit et validé à l'UE dans cette classe. Le résultat attendu est un brouillon, pas encore une transmission.

Dans ces exemples, remplacer chaque matricule par un étudiant réellement présent dans le carnet de cette affectation. Les matricules `21T2355`, `ETU-TEST-02` et `ETU-TEST-03` sont illustratifs et peuvent être rejetés s'ils ne figurent pas dans la classe.

8.3 Import CSV/XLSX

Préparer un CSV d'essai, par exemple :

```
matricule,note,situation
21T2355,14.5,NUMERIQUE
ETU-TEST-02,,ABSENTE
ETU-TEST-03,,MANQUANTE
```

Créer une demande avec la méthode `POST` et ce chemin :

```
{{baseUrl}}/enseignant/affectations/{{assignmentId}}/evaluations/{{evaluationId}}/notes/importer
```

Dans **Body > form-data**, clé `fichier` de type File. Accepter les fichiers `xlsx`, `csv`, `txt`, taille max 10 Mo. Le serveur retourne un aperçu avec `notes`, `erreurs`, `importables`, `total`. Cette route prépare l'aperçu ; elle n'enregistre pas directement les notes. Vérifier les erreurs, puis appeler la route de sauvegarde précédente avec les notes contrôlées.

Tester une ligne avec note 21, matricule inexistant et doublon : ces lignes doivent être signalées comme erreurs d'import. Ne pas confirmer si l'aperçu est incorrect.

8.4 Transmission, bulletin provisoire et retrait logique

Transmettre les brouillons numériques et absences validables :

```
POST {{baseUrl}}/enseignant/affectations/{{assignmentId}}/evaluations/{{evaluationId}}/notes/transmettre
```

Corps :

```
{"motif":"Transmission des notes de recette à la cellule"}
```

Attendu : nombre de notes transmises et message de visibilité dans les bulletins provisoires. Vérifier par `GET /enseignant/affectations/{id}/bulletins-provisoires?periode=S1`, puis avec le compte Étudiant et l'aperçu Agent. La période obligatoire est `S1`, `S2` ou `ANNUELLE`.

Pour retirer une note de l'affichage/calcul, appeler `PATCH /enseignant/affectations/{id}/notes/{note}/retrait` :

```
{"motif":"Correction de la ligne de test demandée", "version_note":1}
```

Le retrait est logique : la note devient `MANQUANTE`, son historique est conservé et le calcul peut signaler `EL`. Il ne s'agit pas d'un effacement physique.

8.5 Requêtes affectées

Lire `GET /enseignant/requetes`. Pour décider une requête affectée par l'Agent, utiliser :

```
PATCH {{baseUrl}}/enseignant/requetes/{{requestId}}
```

```
{"decision":"REJETER", "motif":"Justificatif insuffisant pour ce test"}
```

ou `AUTORISER_EVALUATION` avec un motif. Vérifier l'état depuis l'Agent et l'Étudiant. Tester une requête non affectée : l'API doit la refuser.

9 Parcours 3 – Étudiant

9.1 Profil, inscription et référentiels

Avec le jeton Étudiant, tester `GET /auth/me`, `GET /profil/etudiant` et `GET /referentiels/annees-academiques`. Le compte doit avoir une inscription de classe existante pour les opérations métier (fiche d'inscription, UE, notes). Le seeder ne garantit pas cette inscription.

Lire les choix du semestre : `GET /etudiant/inscriptions?semestre=1`. Enregistrer les UE par code :

```
PUT {{baseUrl}}/etudiant/inscriptions
{
  "semestre": 1,
```

```
  "codes_ue": ["INF301", "INF302"]
}
```

Soumettre : `POST /etudiant/inscriptions/soumettre` avec `{"semestre":1}`. Une soumission sans UE doit retourner `422`. Suivre ensuite le statut côté Agent. Une inscription déjà validée ne peut plus être modifiée de la même façon.

9.2 Résultats, notes provisoires et requêtes

Lire `GET /etudiant/resultats/provisoires`. Le résultat doit être évolutif et signalé comme provisoire. Lire `GET /etudiant/resultats?periode=S1` pour les bulletins publiés seulement. Les deux routes n'ont pas le même statut : ne pas confondre aperçu provisoire et version officielle figée.

Créer une requête avec `POST /etudiant/requetes`, JSON :

```
{
  "code_ue": "INF301",
  "type": "Autre demande",
  "message": "Je demande la vérification de cette situation dans le cadre du test."
}
```

Valeurs de `type` : `Vérification de note`, `Absence justifiée`, `Autre demande`. Une vérification/contestation de note nécessite un bulletin publié. Une absence justifiée impose un fichier `justificatif` en PDF/JPG/PNG de 5 Mo maximum : utiliser **form-data**, clé `justificatif`, avec les champs texte `code_ue`, `type`, `message`. Lire `GET /etudiant/requetes` ; le téléchargement autorisé se fait par `GET /etudiant/requetes/{id}/justificatif`. Les requêtes créées par cette route ne contiennent pas encore l'EC concerné, ce qui bloque leur transmission à un enseignant dans l'état actuel du backend.

9.3 Notifications, profil et rattrapages

Tester `GET /etudiant/notifications`, puis `PATCH /etudiant/notifications/{id}/lue`. Mettre à jour le courriel avec `PUT /etudiant/profil` et `{"courriel":"test.gnu@example.test"}`. Lire `GET /etudiant/rattrapages` après publication d'un résultat et création d'une session.

10 Tester l'application dans le navigateur

Postman confirme la réponse de l'API ; l'interface confirme que l'utilisateur peut réellement effectuer le parcours. Garder les deux serveurs actifs. Ouvrir `http://127.0.0.1:8780`, choisir l'acteur, saisir ses identifiants puis cliquer sur le bouton de connexion. Pour passer à un autre rôle, se déconnecter et saisir le login de cet autre compte. Ne pas réutiliser une session d'un rôle précédent.

10.1 Espace Agent – cellule informatique

Après rattachement du compte Agent à Informatique, vérifier chaque entrée de navigation :

1. **Vue d'ensemble** : les compteurs du tableau de bord sont cohérents avec l'API `/agent/dashboard`.
2. **Structure & catalogue** : consulter les classes, années, filières/niveaux et catalogue d'UE/EC ; créer une classe/UE seulement dans le périmètre de test.
3. **Comptes & affectations** : consulter les comptes du département, créer une affectation valide et vérifier qu'elle arrive chez le bon enseignant.
4. **Inscriptions** : contrôler le statut d'une soumission, valider ou refuser un dossier de test avec un motif.
5. **Requêtes étudiantes** : examiner une requête et tenter son routage. Dans le code actuel, les requêtes déposées par l'Étudiant n'ont pas d'EC (‘matiere_id‘ vide), ce qui empêche le routage ; consigner le scénario comme bloqué.
6. **Sessions de rattrapage** : créer et faire évoluer une session de test.
7. **Bulletins & publications** : comparer l'aperçu provisoire avec les notes saisies ; laisser la publication officielle désactivée pendant la recette ordinaire.

8. **Journal d'audit** : retrouver les actions précédentes et leurs motifs.

10.2 Espace Enseignant

Le compte de démonstration doit d'abord avoir une affectation active. Dans **Mes affectations**, ouvrir l'EC de test. Paramétrer le plan d'évaluation, contrôler le total 100 %, ouvrir le calendrier, passer une évaluation à réalisée, saisir/importer et enregistrer les notes. Vérifier l'état après rechargement, transmettre les notes et observer le bulletin provisoire. Tester une note manquante, une absence, une valeur limite (0, 20) et une valeur hors limite (21). Ouvrir ensuite les requêtes affectées ; le parcours de routage dépend de la limite signalée ci-dessus.

10.3 Espace Étudiant

Le compte doit être lié à une inscription de classe et à des UE accessibles. Vérifier dans l'ordre :

1. **Vue d'ensemble** : identité, niveau, classe et année visibles.
2. **Notes & bulletins** : résultat provisoire qui évolue après transmission ; résultat officiel visible séparément après publication.
3. **Inscription pédagogique** : consulter le semestre, choisir les UE permises, soumettre puis constater le statut Agent.
4. **Mes requêtes** : créer une demande normale ; utiliser un justificatif uniquement pour tester le formulaire d'absence justifiée.
5. **Mes rattrapages** : constater l'éligibilité et les sessions ouvertes lorsque des résultats publiés le permettent.
6. **Profil & notifications** : lire les notifications, marquer une notification lue, modifier un courriel de test.

Si un bouton n'est pas visible ou une page affiche une erreur, relever le rôle, l'URL de la page, l'heure et le message. Dans Postman, appeler la route équivalente : si l'API répond correctement mais pas l'écran, l'anomalie est probablement côté interface ; si les deux échouent, conserver aussi le code HTTP et le corps de réponse.

11 Tests d'autorisation et de cohérence

Ces cas doivent être reproduits avec les jetons des acteurs correspondants :

1. Sans Bearer token, `GET /agent/dashboard` retourne `401`.
2. Jeton Étudiant sur `GET /agent/dashboard` retourne un refus de rôle, typiquement `403`.
3. Agent sans rattachement : la lecture est vide et aucune classe étrangère ne peut être modifiée.
4. Agent rattaché Informatique : tenter un ID d'un autre département ; réponse non autorisée ou non trouvée (404), aucune modification appliquée.
5. Enseignant A tente l'`assignmentId` de l'enseignant B ; accès refusé/non trouvé.
6. Saisie de note hors intervalle, mauvais matricule, poids qui ne totalise pas 100 ou transition de statut impossible ; réponse de validation/conflit (422/409).
7. Publication sans habilitation ; réponse attendue 403. Ne pas ajouter l'habilitation en base pour simplement contourner ce test.
8. Jeton expiré ou déconnexion ; la route protégée doit refuser le jeton.

La réponse de refus doit être contrôlée avec l'absence d'effet de bord : relire la ressource concernée après le test.

12 Catalogue synthétique des routes

Toutes les routes ci-dessous sont préfixées par `{{baseUrl}}`. Les mutations doivent être d'abord exercées sur des données de test.

Dans les chemins ci-dessous, remplacer un segment entre accolades par son ID numérique réel. Tous les chemins sont préfixés par `{{baseUrl}}` et requièrent un Bearer token sauf la connexion.

```
COMMUN
POST /auth/login
GET /auth/me
POST /auth/logout
POST /auth/logout-all
PUT /auth/password
GET /referentiels/annees-academiques
GET /referentiels/filieres
GET /referentiels/specialites
GET /referentiels/niveaux
GET /referentiels/semestres
GET /referentiels/classes
GET /referentiels/ues
GET /referentiels/ecs

AGENT
GET /agent/dashboard
GET /agent/classes
POST /agent/classes
GET /agent/niveaux
GET /agent/catalogue
POST /agent/ues
POST /agent/ues/{ue}/elements
PATCH /agent/ues/{ue}/statut
GET /agent/comptes
PATCH /agent/comptes/{utilisateur}/statut
GET /agent/affectations
POST /agent/affectations
PATCH /agent/affectations/{affectation}
GET /agent/inscriptions
PATCH /agent/inscriptions/{inscriptionSemestre}
GET /agent/requetes
PATCH /agent/requetes/{requete}/affectation
GET /agent/classes/{classe}/apercu-bulletins?periode=S1
POST /agent/classes/{classe}/publications
GET /agent/audit
GET /agent/rattrapages/sessions
POST /agent/rattrapages/sessions
PATCH /agent/rattrapages/sessions/{session}

ENSEIGNANT
GET /enseignant/affectations
GET /enseignant/affectations/{id}/plan-evaluation
PUT /enseignant/affectations/{id}/plan-evaluation
GET /enseignant/affectations/{id}/carnet
GET /enseignant/affectations/{id}/bulletins-provisoires?periode=S1
PATCH /enseignant/affectations/{id}/evaluations/{evaluation}
PUT /enseignant/affectations/{id}/evaluations/{evaluation}/notes
POST /enseignant/affectations/{id}/evaluations/{evaluation}/notes/importer
POST /enseignant/affectations/{id}/evaluations/{evaluation}/notes/transmettre
PATCH /enseignant/affectations/{id}/notes/{note}/retrait
GET /enseignant/requetes
PATCH /enseignant/requetes/{requete}

ETUDIANT
GET /etudiant/inscriptions?semestre=1
PUT /etudiant/inscriptions
POST /etudiant/inscriptions/soumettre
GET /etudiant/resultats
GET /etudiant/resultats/provisoires
GET /etudiant/rattrapages
GET /etudiant/requetes
POST /etudiant/requetes
GET /etudiant/requetes/{requete}/justificatif
GET /etudiant/notifications
PATCH /etudiant/notifications/{notification}/lue
PUT /etudiant/profil
```

Pour le détail des champs requis de chaque mutation, utiliser d'abord l'exemple du parcours correspondant et le message de validation renvoyé en cas de champ manquant. Le backend peut faire évoluer ces champs ; vérifier les règles actuelles avant d'écrire un script d'automatisation pérenne.

13 Règles métier à vérifier pendant la recette

— Les notes de contrôle sont saisies sur 20 ; le calcul convertit vers le barème officiel sur 100.

— Les pondérations du plan doivent totaliser 100 %. Cas d'essai : CC 30 + SN 70 ; CC 20 + TP 30 + SN 50.

— La grille confirmée : A 80+ = 4.00 ; A- 75–79 = 3.70 ; B+ 70–74 = 3.30 ; B 65–69 = 3.00 ; B- 60–64 = 2.70 ; C+ 55–59 = 2.30 ; C 50–54 = 2.00 ; C- 45–49 = 1.70 ; D+ 40–44 = 1.30 ; D 35–39 = 1.00 ; E 30–34 = 0 ; F 0–29 = 0.

— Arrondir la note finale d'UE puis la MGP au centième usuel. Garder la précision complète au cours du calcul.

— Note manquante : situation éliminatoire (EL), prise en compte dans le calcul selon le barème institutionnel confirmé ; la présence d'un échec n'empêche pas à elle seule la production du bulletin.

— Une MGP sous le seuil institutionnel 2.00 porte la décision d'échec dans le PV ; vérifier qu'un bulletin est néanmoins produit selon la configuration validée.

— Les bulletins provisoires évoluent avec les notes transmises. Une publication officielle fige une version historique.

Comparer des résultats calculés à la fois dans l'API et dans les interfaces Agent, Enseignant, Étudiant. Toute divergence de note, crédit, statut, MGP ou version doit être consignée comme anomalie.

14 Plan de recette et critères d'acceptation

Lot	Critère de sortie
Infrastructure	PostgreSQL répond, migrations à jour, API et Flutter accessibles en local.
Comptes	Trois logins testés ; rôle retourné conforme ; jeton conservé localement ; déconnexion testée.
Périmètre	Agent voit le département attribué et ne peut modifier un autre périmètre.
Référentiel	Classe, année, niveau, UE/EC lus sans doublon ni filière interdite.
Enseignement	Affectation compatible, plan totalisant 100 %, saisie/import contrôlés, transmission et historique observés.
Étudiant	Inscription soumise, décision suivie, résultats provisoires/publicés distingués, requête créée et suivie.
Calcul	Grades, points, crédits, note EL, MGP arrondie et décision recoupés.
Publication	Prévisualisation vérifiée ; publication testée seulement sur classe isolée avec autorisation explicite ; nouvelle version contrôlée si nécessaire.
Audit	Actions administratives et pédagogiques de test retrouvées avec motif et date.

Le test est **PASS** seulement si le résultat réel correspond à l'attendu et que les données ont l'état attendu après relecture. Utiliser **FAIL** si le comportement est incorrect, **BLOQUÉ** si une précondition ou permission manque, **NON TESTÉ** sinon.

14.1 Tests automatisés facultatifs

Ces commandes complètent la recette manuelle. Elles ne vérifient pas à elles seules les écrans avec un utilisateur, les permissions réelles de la base active ni tous les échanges Postman.

Tests Flutter :

```
cd "/home/agent-47/Téléchargements/GNU---Gestion-des-notes-Universitaires/frontend"
flutter test
```

Tests Laravel :

```
cd "/home/agent-47/Téléchargements/GNU---Gestion-des-notes-Universitaires/backend"
php artisan test
```

Le fichier `backend/phpunit.xml` sélectionne une base nommée `gnu_notes_test`. Avant d'exécuter les tests Laravel, confirmer que cette base est bien une base jetable de test et non une copie de travail contenant des données utiles. Ne jamais remplacer ce nom par `gnu_notes` pour faire passer les tests. Si les dépendances ou le schéma de test sont absents, la commande peut échouer ; noter l'erreur, ne pas modifier la base locale de recette pour la contourner.

14.2 Scénarios numérotés à exécuter

Exécuter les lignes dans l'ordre. Les tests **BLOQUÉ** dépendent des jeux de données absents ou de la limite d'API signalée ; les noter ainsi plutôt que d'inventer des IDs ou de modifier des données réelles.

ID	Action	Résultat attendu
PRE-01	Démarrer PostgreSQL et contrôler le port 5435.	`accepting connections`.
PRE-02	Consulter `php artisan migrate:status`.	Migrations attendues à `Ran`.
PRE-03	Démarrer l'API port 8000 et Flutter port 8780.	Deux serveurs affichent leur URL.
PRE-04	Requête sans token sur `GET /api/v1/agent/dashboard`.	`401` ; API joignable.
AUTH-01	Connexion Agent.	`200`, rôle `AGENT`, token reçu.
AUTH-02	Connexion Enseignant.	`200`, rôle `ENSEIGNANT`.
AUTH-03	Connexion Étudiant.	`200`, rôle `ETUDIANT`.
AUTH-04	Connexion avec mot de passe erroné.	`401`, aucune session reçue.
AUTH-05	Appeler `/auth/me` avec chaque token.	Profil correspondant au rôle connecté.
AUTH-06	Utiliser le token étudiant sur une route Agent.	`403`, pas de donnée ni mutation.
AUTH-07	Déconnexion, puis réutilisation du token.	`204`, token révoqué/refusé après déconnexion.
AG-01	Dashboard Agent avant rattachement de département.	Alerte/périmètre vide ; comportement compris.
AG-02	Rattacher Informatique et recharger dashboard.	Département et compteurs correspondants visibles.
AG-03	Lister classes et niveaux.	Seules les données du périmètre sont visibles.
AG-04	Lister catalogue UE/EC.	Code, crédits, semestre et état cohérents.
AG-05	Ouvrir la classe de test dans l'interface.	Niveau, année, version programme cohérents.
AG-06	Créer une classe avec IDs du périmètre.	`201` ; ID réutilisable dans Postman.
AG-07	Créer une classe avec niveau hors département.	Refus ; classe non créée.
AG-08	Créer une UE de test en brouillon.	`201`, statut `BROUILLON`.
AG-09	Activer UE avec zéro EC ou plus de deux EC.	Refus `422`.
AG-10	Ajouter EC puis activer UE de test.	Succès avec un/deux EC ; statut `ACTIF`.
AG-11	Créer affectation enseignant/classe/EC compatibles.	`201` ; visible côté enseignant.

ID	Action	Résultat attendu
AG-12	Créer affectation incompatible ou enseignant inactif.	Refus `422` ; aucun lien parasite.
AG-13	Suspendre son propre compte Agent.	Refus `422`.
AG-14	Valider une inscription `SOUMISE` de test.	`200` ; statut `VALIDEE`.
AG-15	Reprendre une inscription déjà décidée.	Conflit `409` ; état inchangé.
AG-16	Router une requête créée par l'étudiant.	**BLOQUÉ connu** : la requête arrive sans EC associé.
AG-17	Créer session de rattrapage à date future.	`201` ; état `PREVUE`.
AG-18	Faire évoluer session `PREVUE -> OUVERTE -> CLOTUREE`.	Transitions autorisées ; transition interdite refusée.
AG-19	Lire aperçu de bulletin S1/S2/annuel.	MGP/UE/EL visibles ; aucun gel officiel.
AG-20	Publier sans habilitation Agent distincte.	`403` ; aucune publication créée.
AG-21	Vérifier journal d'audit après mutations.	Acteur, date, objet et motif retrouvés.
ENS-01	Lire les affectations actives du professeur.	Affectation de test présente.
ENS-02	Enregistrer un plan totalisant 100 %.	`201` ; nouvelle version active.
ENS-03	Enregistrer plan totalisant 90 %.	Refus `422` ; version courante intacte.
ENS-04	Lire le carnet et marquer une évaluation réalisée.	Étudiants/notes chargés ; statut réalisé.
ENS-05	Saisir note 0 puis note 20.	Valeurs acceptées et sauvegardées.
ENS-06	Saisir note 21, mauvais matricule ou doublon.	Refus/erreur d'import ; aucune ligne incorrecte.
ENS-07	Importer CSV et examiner l'aperçu.	Lignes importables et erreurs séparées.
ENS-08	Sauvegarder puis relire les brouillons.	Nombre et versions de notes cohérents.
ENS-09	Transmettre les notes.	Notes validées ; audit ; bulletin provisoire actualisé.
ENS-10	Retirer une note avec version courante et motif.	Situation manquante ; historique conservé.
ENS-11	Retirer avec ancienne version.	`409` ; aucune mise à jour.
ENS-12	Ouvrir les bulletins provisoires de l'affectation.	Statut `PROVISOIRE`, données à jour.
ENS-13	Traiter requête transmise par Agent.	**BLOQUÉ** si la requête n'a pas l'EC requis.
ETU-01	Lire profil et fiche d'inscription.	Identité/classe/année du compte courant.
ETU-02	Lire les UE proposées pour un semestre.	Unités actives du niveau et programme.
ETU-03	Enregistrer choix d'UE connus.	Brouillon mis à jour.
ETU-04	Enregistrer un code UE non proposé.	Refus `422` ; sélection inchangée.
ETU-05	Soumettre une sélection non vide.	État `SOUMISE` ; Agent la voit.

ID	Action	Résultat attendu
ETU-06	Soumettre sans UE.	Refus `422`.
ETU-07	Lire résultats provisoires après transmission enseignant.	Statut provisoire ; note actualisée.
ETU-08	Lire résultat publié, si classe publiée.	Version officielle distincte, datée.
ETU-09	Déposer requête « Autre demande ».	`201` ; requête retrouvée dans sa liste.
ETU-10	Déposer absence justifiée sans fichier.	Refus `422`.
ETU-11	Déposer absence avec PDF/JPG/PNG de test.	`201` ; fichier accessible au propriétaire.
ETU-12	Lire/marquer notification lue.	État `lue=true`.
ETU-13	Lire ses rattrapages après bulletin publié.	Seules candidatures éligibles/connues visibles.
MGP-01	Tester notes de part et d'autre de chaque seuil.	Grade et points conformes à la grille officielle.
MGP-02	Tester note manquante et absence.	Statut EL/échec selon la règle institutionnelle.
MGP-03	Tester MGP exactement 2.00 puis sous 2.00.	Décision annuelle conforme au seuil.
MGP-04	Comparer l'arrondi API et bulletin.	UE et MGP au centième, valeur concordante.
MGP-05	Contrôler bulletin d'un étudiant en échec.	Bulletin produit avec décision échec.

15 Fiche d'exécution à dupliquer

```
ID du test :
Date / commit ou version de l'application :
Rôle / login (ne jamais noter le mot de passe) :
Outil : Postman / interface / SQL lecture seule
Nom de la requête Postman :
Préconditions et identifiants de données :
Étapes envoyées :
Résultat attendu :
Code HTTP et résumé de la réponse :
Vérification après action / état final :
Statut : PASS / FAIL / BLOQUE / NON TESTE
Capture (jeton et données personnelles masqués) :
Anomalie / suivi / responsable :
```

15.1 Tableau de progression

Domaine	Prévu	PASS	FAIL	Bloqué
Connexion/rôles	6			
Agent – structure/catalogue/classes	10			
Agent – affectations/inscriptions/requêtes	10			
Agent – rattrapages/bulletins/audit	10			
Enseignant – plan/carnet/notes/import	12			
Étudiant – inscription/résultats/requêtes	12			
Calcul MGP et règles d'arrondi	8			
Sécurité/périmètre	8			

16 Dépannage rapide

— **Connexion refusée** : vérifier base, login, mot de passe, état actif, rôle choisi, profil complet, expiration du jeton. Le seeder ne garantit pas les relations pédagogiques.

— **Dashboard Agent vide** : contrôler la ligne dans `gnu.agent_departement` et vérifier qu'il existe des classes/filières/années dans ce département.

— **Aucune affectation enseignant** : la créer côté Agent en associant enseignant actif, EC du programme, classe et version cohérents.

— **Aucun étudiant dans le carnet** : confirmer inscription de classe validée, inscription UE validée et appartenance à la classe.

— **403** : authentification réussie mais rôle ou habilitation manquante. Pour publication, le droit distinct est requis.

— **404** : ressource inexistante ou hors périmètre. Vérifier l'ID retourné par la lecture, ne pas deviner un identifiant.

— **409** : vérifier état courant, verrouillage d'une évaluation notée, version de note ou publication déjà existante.

— **422** : lire les erreurs de validation ; confirmer clés JSON, dates, valeurs d'énumération, matricule et total des poids.

— **Postman conserve une ancienne session** : exécuter `POST /auth/logout`, vider la variable token et refaire la connexion.

17 Construire la documentation utilisateur d'environ 50 pages

Après la recette, transformer les parcours confirmés en manuel avec captures et résultats réels. Proposition de répartition :

Partie	Contenu	Pages indicatives
1	Présentation, architecture et vocabulaire LMD	4
2	Installation, accès et sécurité des comptes	5
3	Parcours Agent, classes, catalogue et affectations	12
4	Parcours Enseignant, évaluations, saisie et transmission	10
5	Parcours Étudiant, inscriptions, bulletins et requêtes	8
6	Calcul MGP, rattrapages, publication et audit	6
7	Postman, scénarios de recette, dépannage et annexes	5

Pour chaque page d'écran, décrire son but, les rôles habilités, les prérequis, les champs, le geste utilisateur, les statuts, les erreurs possibles et la preuve observée. Indiquer la version du logiciel et la date de validation. Une capture d'un écran ne doit jamais révéler un jeton, un mot de passe, un justificatif ou une donnée personnelle non anonymisée.

18 Sources techniques vérifiées

Ce guide a été construit à partir des routes et contrôleurs d'authentification, Agent, Enseignant et Étudiant, du seeder des comptes de démonstration, ainsi que des migrations d'authentification et de bulletin progressif. Les chemins sont relatifs à la racine du dépôt GNU.

Les noms de routes, champs et statuts doivent être revérifiés dans le code lors d'une évolution de l'API. La recette de production n'est pas réputée effectuée parce que ce guide a été produit.